

QUESTIONS

Q1
Web Browser Back Navigation
10 marks**Q2**
University Course Enrollment
Manager
10 marks**Q3**
Railway Parcel Dispatch
Tracking System
10 marks

3/3 answered

Q1 Web Browser Back Navigation

10 marks

A web browser stores recently visited pages so that users can navigate backward.

Requirements:

Add visited page URLs using push()

Perform the Back operation using pop()

Display the current page using peek()

Your Code

```
1 import java.util.Stack;
2 import java.util.Scanner;
3 public class BrowserBackNavigation {
4     public static void main(String[] args) {
5         Stack<String> history = new Stack<>();
6         Scanner sc = new Scanner(System.in);
7         history.push("google.com");
8         history.push("wikipedia.org");
9         history.push("simatslab.saveetha.com");
10        history.push("github.com");
11        System.out.println("Current Page: " + history.peek());
12        System.out.println("\nGoing Back");
13        history.pop();
14        System.out.println("Current Page: " + history.peek());
15        System.out.println("\nGoing Back");
16        history.pop();
17        System.out.println("Current Page: " + history.peek());
18        System.out.println("\nRemaining Browser History:");
19        for (String url : history) {
20            System.out.println(url);
21        }
22        sc.close();
23    }
24 }
```

Run

Save

Input

stdin input

Output

Current Page: github.com

Going Back
Current Page:
simatslab.saveetha.com

QUESTIONS

Q1
Web Browser Back Navigation
10 marks**Q2**
University Course Enrollment
Manager
10 marks**Q3**
Railway Parcel Dispatch
Tracking System
10 marks

3/3 answered

Q2 University Course Enrollment Manager

10 marks

A university maintains a list of students enrolled in a course. Students who withdraw must be removed from the list.

Requirements:

Add student names to an ArrayList

Traverse the list using an Iterator

Remove withdrawn students and display the updated enrollment list

Your Code

```
1 import java.util.ArrayList;
2 import java.util.Iterator;
3 public class UniversityCourseEnrollmentManager {
4     public static void main(String[] args) {
5         ArrayList<String> enrolledStudents = new ArrayList<>();
6         enrolledStudents.add("Aarav");
7         enrolledStudents.add("Priya");
8         enrolledStudents.add("Rohan");
9         enrolledStudents.add("Sneha");
10        enrolledStudents.add("Kabir");
11        System.out.println("Initial Enrollment List:");
12        for (String student : enrolledStudents) {
13            System.out.println(student);
14        }
15        ArrayList<String> withdrawnStudents = new ArrayList<>();
16        withdrawnStudents.add("Rohan");
17        withdrawnStudents.add("Sneha");
18        Iterator<String> iterator = enrolledStudents.iterator();
19        while (iterator.hasNext()) {
20            String currentStudent = iterator.next();
21            if (withdrawnStudents.contains(currentStudent)) {
22                iterator.remove();
23                System.out.println("\nRemoved (Withdrawn): " + currentStudent);
24            }
25        }
26        System.out.println("\nUpdated Enrollment List:");
27        for (String student : enrolledStudents) {
28            System.out.println(student);
29        }
30    }
31 }
```

Input

stdin input

Output

```
Initial Enrollment List:
Aarav
Priya
Rohan
Sneha
Kabir
```


QUESTIONS

Q1

Web Browser Back Navigation
10 marks

Q2

University Course Enrollment
Manager
10 marks

Q3

Railway Parcel Dispatch
Tracking System
10 marks

3/3 answered

Q3 Railway Parcel Dispatch Tracking System

10 marks

A railway parcel office loads parcels onto a dispatch container. The last loaded parcel must be unloaded first, and parcel details must be searchable.

Requirements:

Store parcel IDs in a Stack

Store parcel ID and destination in a HashMap

Unload parcels using pop() and display the destination of the unloaded parcel

Your Code

```
1 import java.util.*;
2 public class RailwayParcelDispatch {
3     public static void main(String[] args) {
4         Stack<Integer> parcelStack = new Stack<>();
5         HashMap<Integer, String> parcelMap = new HashMap<>();
6         parcelStack.push(101);
7         parcelMap.put(101, "Chennai");
8         parcelStack.push(102);
9         parcelMap.put(102, "Bangalore");
10        parcelStack.push(103);
11        parcelMap.put(103, "Hyderabad");
12        parcelStack.push(104);
13        parcelMap.put(104, "Mumbai");
14        System.out.println("Parcels loaded onto the dispatch container:");
15        System.out.println(parcelStack);
16        System.out.println("\nUnloading parcels (Last In, First Out):");
17        while (!parcelStack.isEmpty()) {
18            int unloadedParcelId = parcelStack.pop();
19            String destination = parcelMap.get(unloadedParcelId);
20            System.out.println("Unloaded Parcel ID: " + unloadedParcelId);
21        }
22        System.out.println("\nAll parcels have been unloaded and dispatched.");
23    }
24 }
```

Input

stdin input

Output

```
Parcels loaded onto the dispatch
container:
[101, 102, 103, 104]
```

```
Unloading parcels (Last In,
First Out):
```